



Logout

[Help Center](#)[PLEXOS Cloud](#)[PLEXOS Desktop](#)[PLEXOS Cloud](#) > [PLEXOS Cloud](#) > [Simulations](#)

Export as PDF



Pre- and Post-Simulation Tasks

Overview

Pre- and Post-Simulation Tasks enable users to execute arbitrary logic on Energy Exemplar's compute infrastructure before or after a simulation. This significantly reduces run times by eliminating file transfers and leveraging cloud computing at scale. Pre- and post-simulation tasks are currently supported only on Linux environments. Multiple pre- and post-simulation tasks can be executed simultaneously, and several valuable tasks are already available on the Energy Exemplar Marketplace.

All tasks are executed independently, ensuring a secure Energy Exemplar infrastructure and maintaining a consistent user environment. Tasks are executed according to the Execution Order property and can be configured to be skipped if the simulation fails or if any errors occur.

Pre-requisites

Before setting up and executing scripts, users must familiarize themselves with essential tools and environment concepts that ensure scripts function consistently across simulations. Energy Exemplar supports standalone Linux binaries or Python scripts as long as all required dependencies are explicitly listed in a **requirements.txt** file.

Suggested Tools and Resources

1. **Cloud CLI** The **Energy Exemplar Cloud CLI** is pre-configured with the **Energy Exemplar Python API**. Users do not need to configure environment variables or enter credentials manually, as these steps are pre-configured within the Cloud CLI setup. However, they should verify dependencies are correctly loaded before execution.
2. **DataHub Energy Exemplar DataHub** is the primary storage system for simulation-related files and artifacts in the cloud. All task-related data must be stored and referenced within **DataHub** to ensure seamless access and retrieval during script execution. Familiarity with **DataHub** is essential for the effective use of cloud-based workflows.
3. **DuckDB** [↗](#) **DuckDB** and its [DuckDB Python library](#) [↗](#) enable high-performance querying and aggregation of structured data. **DuckDB** supports multiple tabular file formats, including Parquet, SQLite, and CSV, making it a powerful tool for analyzing and processing simulation data.

Requirements.txt

For Python-based scripts, users must create a requirements.txt file listing all additional dependencies required for execution. Only one requirements.txt file is allowed per Simulation, meaning dependencies from multiple scripts must be consolidated into a single file.

Example Requirements File:

```
duckdb
pandas
matplotlib
```

These dependencies ensure necessary Python libraries are available throughout the script execution process.

Using Pre- and Post-Processing in PLEXOS Cloud

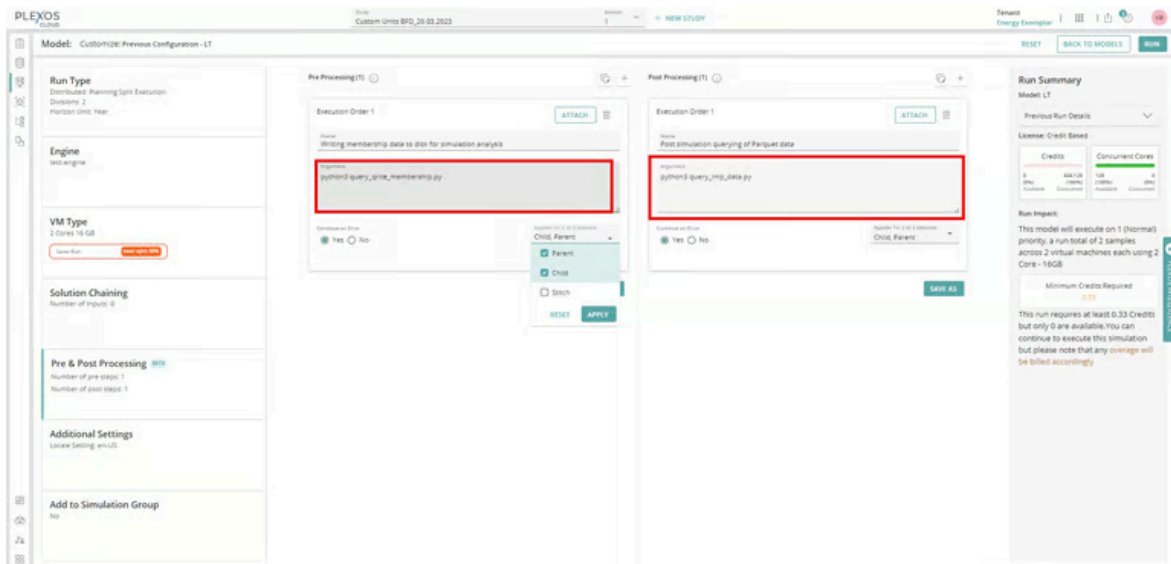
The **Pre- and Post-Processing** section in **PLEXOS Cloud**, allows you to define and manage pre- and post-simulation tasks directly within the **PLEXOS Cloud** interface.

Pre-simulation tasks run **before** the PLEXOS engine starts, while post-simulation tasks run **after** it completes.

Examples include:

- **Pre-simulation tasks** -- Downloading or extracting additional input data.
- **Post-simulation tasks** -- Querying Parquet data, performing ETL, or uploading outputs back to DataHub.

This functionality is available for **Linux VMs** and can be configured from the **Customize** option in the Run Configuration Panel.



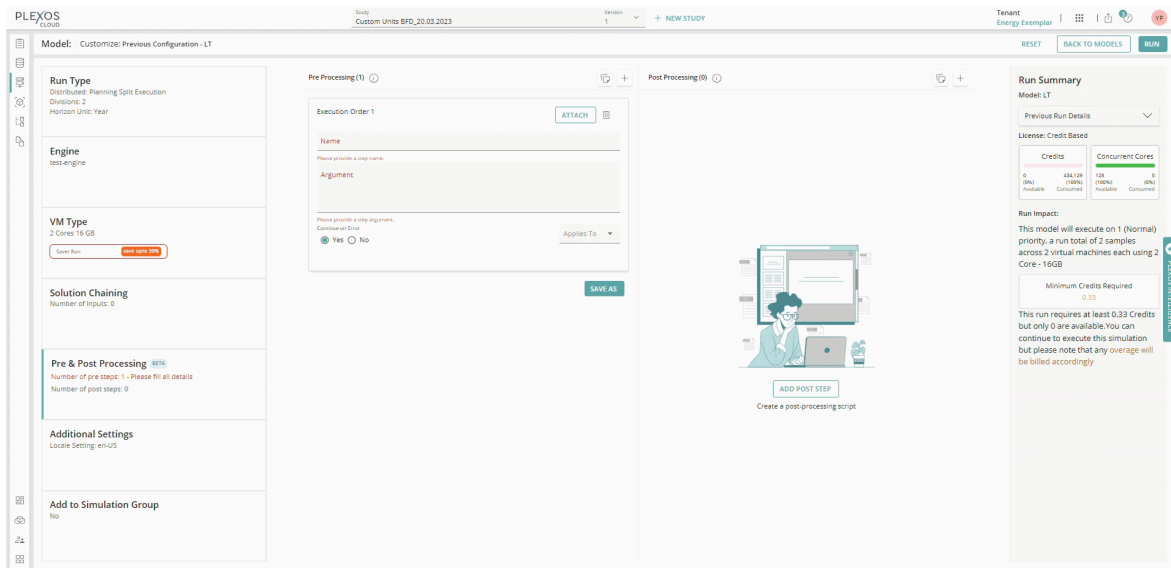
► How It Works

In the Pre- and Post-Processing section, you can add one or multiple steps. Each step requires the following details:

1. **Step Name:** A descriptive name for the task.
2. **Example:** Downloading or extracting additional input data.
3. **Step Argument:** The script or command argument to execute.
4. **Example:** python3 query_imp_data.py
5. **Continue on Error:** A checkbox that indicates whether the run should proceed if the step fails.
6. **Applies To:** Available for **Distributed Run Types** only. Lets you specify if the step applies to the **Parent Simulation**, **Child Simulation**, or **Stitch phase**.
7. **Attach:** Option to attach supporting files from **DataHub** for pre- or post-processing.

Refer to the following GIF for an example:

NOTE: Pre- and post-processing tasks are not available for Gurobi Tuner and Virtual Machine Selection run types.



Execution Flow

When you run a simulation with pre- and post-processing tasks, execution follows this order:

Single Runs

1 Run pre-simulation tasks

PLEXOS Cloud executes all configured pre-simulation tasks first.

2 Run the model

The PLEXOS engine runs the model.

3 Run post-simulation tasks

PLEXOS Cloud executes all configured post-simulation tasks.

4 Generate outputs

After task execution completes, outputs are generated.

Distributed Runs

If you define pre- and post-processing steps for distributed runs by using the **Applies To** setting, execution follows this order:

1 Run parent pre-simulation tasks

PLEXOS Cloud executes pre-simulation tasks assigned to the **Parent Simulation**.

2 Run the parent model

The PLEXOS engine runs the model using the selected distributed run type.

3 Run parent post-simulation tasks

PLEXOS Cloud executes post-simulation tasks assigned to the **Parent Simulation**.

4 Run child pre-simulation tasks

PLEXOS Cloud executes pre-simulation tasks assigned to the **Child Simulation**.

5 Run the child model

The PLEXOS engine runs the child simulation using the selected distributed run type.

6 Run child post-simulation tasks

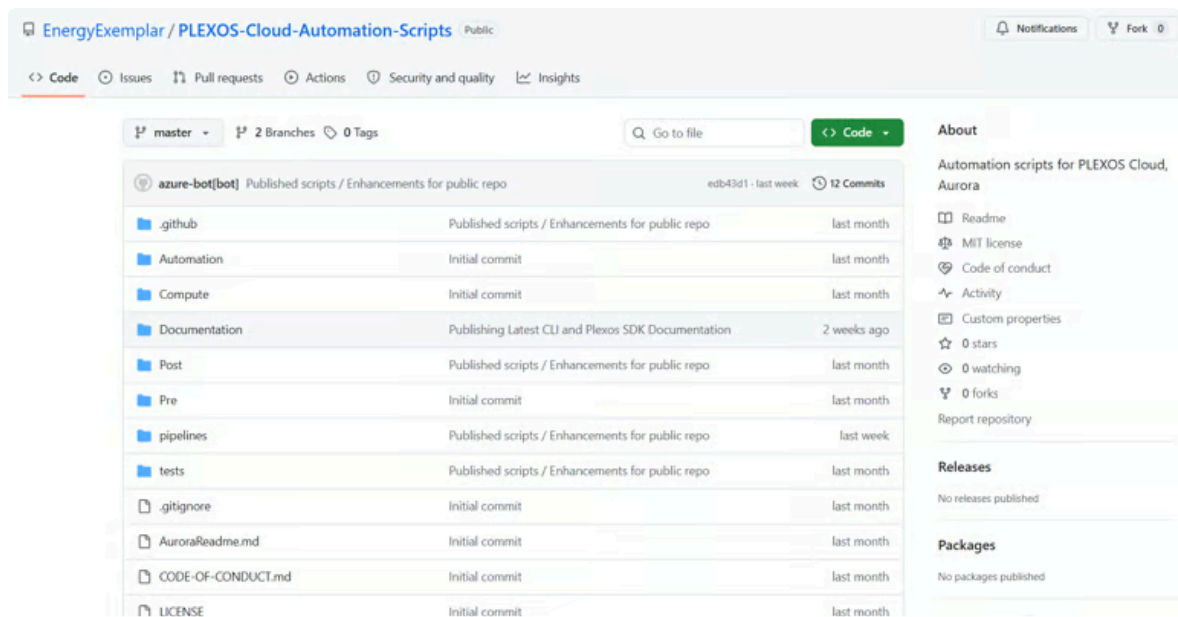
PLEXOS Cloud executes post-simulation tasks assigned to the **Child Simulation**.

Using Public GitHub Automation Scripts for Pre- and Post-Simulation Tasks

PLEXOS Cloud provides a **public GitHub repository** with ready-to-use automation scripts for **PLEXOS** and **Aurora** workflows. This repository includes **pre-simulation**, **post-simulation**, and **standalone scripts** that support common automation scenarios.

These scripts are designed to work **with Pre- and Post-Simulation Tasks**, which allow execution of custom logic on Energy Exemplar's compute infrastructure before or after a simulation run.

This capability supports tasks such as data preparation, validation, transformation, and post-processing.



The repository integrates with PLEXOS Cloud through the **GitHub Connector**, allowing direct access to version-controlled scripts without manual uploads. Once configured, scripts from the repository can be referenced directly in simulation tasks from the **Run Configuration Panel** or via **CLI**.

This creates a streamlined and reusable workflow for managing automation across simulations.

► How it works

I. Accessing automation scripts from the public GitHub repository The public GitHub repository acts as a source for automation scripts that can be referenced directly within simulation workflows. Instead of creating or uploading scripts manually, you can select and use repository-hosted scripts as part of your pre- and post-simulation tasks.

- Energy Exemplar's Repository Link: [PLEXOS Cloud Automation Scripts](#)

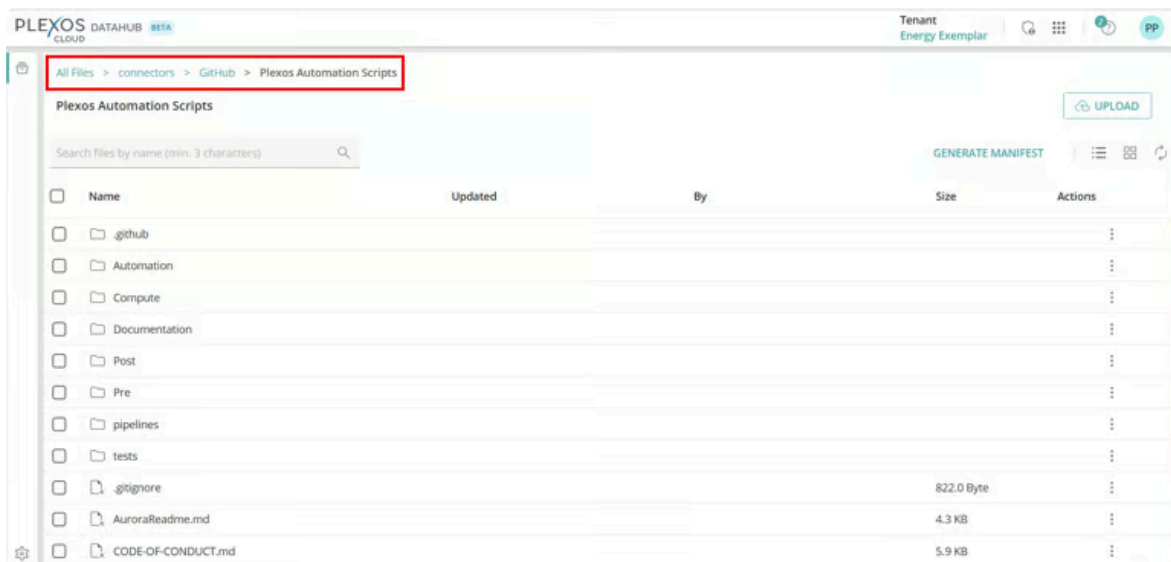
II. Connecting the repository using the GitHub Connector To use scripts from the repository, a GitHub Connector must be configured in DataHub. This connector enables DataHub to access version-controlled assets stored in Git repositories.

Once the connector is created, the repository becomes available as a source when selecting files for simulation tasks.

Example CLI command to create a GitHub connector:

```
pxc datahub connector create \  
  --name "PlexosAutomationScripts" \  
  --connector-type GitHub \  
  --auth-type Pat \  
  --owner "<owner>" \  
  --repository "<repository>" \  
  --branch "<branch>" \  
  --pat ""
```

Refer to the following image for an example:



III. Using scripts in pre and post-simulation tasks Pre- and Post-Simulation Tasks can be configured from the **Run Configuration Panel** or through the CLI.

Within the **Pre and Post Processing** section of the Run Configuration Panel, scripts can be attached to simulation steps. The GitHub repository appears as a selectable source through the configured connector, allowing scripts to be referenced directly without uploading files manually.

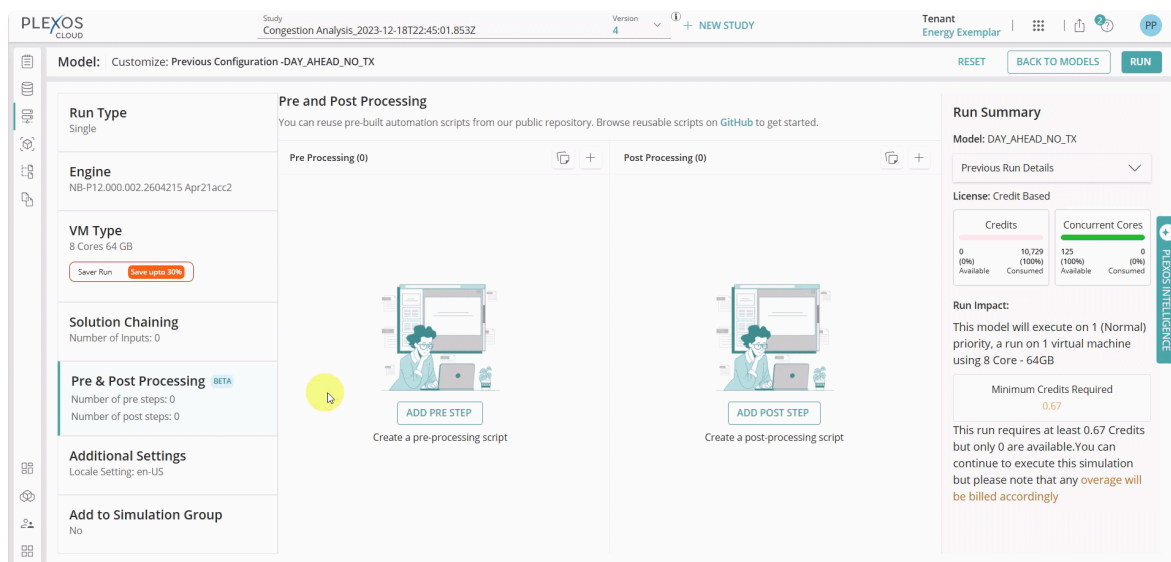
This enables consistent reuse of automation logic across models and simulations while maintaining version control through the repository.

► Example scenario

A model requires preprocessing to prepare input data and post-processing to format outputs for downstream analysis. Instead of uploading scripts manually for each run, a GitHub Connector is configured once and linked to the public repository.

The required scripts are then selected directly in the Pre and Post Processing section of the Run Configuration Panel. This setup allows the same scripts to be reused across multiple simulations while ensuring consistency and reducing setup effort.

Refer to the following GIF for an example:



Script Environment

Scripts are executed in a virtual environment based on **Ubuntu Linux 22.04 LTS**, with two Python interpreters, **Python 3.10** (`python3`) and **Python 3.12** (`python3.12`), installed side by side, along with **Pip** and **Wheel** preinstalled. Including Python dependencies in a `requirements.txt` file ensures they are available for scripts. A single `requirements.txt` file is supported per simulation, so if multiple scripts require different dependencies, they must be combined into one file. Each task runs in a separate container image but shares the same virtual disk mounts. For details on choosing between the two interpreters, see [Selecting the Python Version](#).

Selecting the Python Version

Pre- and post-simulation tasks run in a container with **two Python interpreters installed side by side**. Both Python 3.10 and Python 3.12 are supported for pre- and post-simulation execution. The important point is not that every task uses one fixed version, but that the correct interpreter is selected explicitly for the script being executed.

Command	Version	When to use it
<code>python3</code>	3.10.12	The default interpreter. Use it when the script is compatible with Python 3.10.
<code>python3.12</code>	3.12.13	Use it when the script has been updated for Python 3.12 or the workflow requires Python 3.12 compatibility.

Python 3.12 is added when the container image is built, so simulations do not contact any external archive during execution.

► Selecting the interpreter

The interpreter is defined as part of each task's **Arguments**, so it is chosen individually for every task. Two tasks in the same simulation can use different Python versions. Follow these guidelines when configuring the interpreter:

1. If the script is validated on Python 3.10, run it with `python3`.
2. If the script is validated on Python 3.12, run it with `python3.12`.
3. Always specify the interpreter explicitly. Avoid using the bare `python` command, as the version it resolves to is not guaranteed when multiple Python versions are installed.
4. Keep the interpreter aligned with the version the script has been validated against, to avoid import or syntax errors during execution.

Each interpreter is called in the same way in a task's **Arguments**, with `python3` or `python3.12` preceding the script name:

```
python3 Pre/some_script.py  
python3.12 Post/some_script.py
```

The following example shows two tasks in the same simulation using different interpreters:

```

"SimulationTasks": [
{
  "Name": "Prepare input data",
  "TaskType": "Pre",
  "Files": [
    { "Path": "Team/User/prepare_inputs.py", "Version": null },
    { "Path": "Team/User/requirements.txt", "Version": null }
  ],
  "Arguments": "python3 prepare_inputs.py",
  "ContinueOnError": false,
  "ExecutionOrder": 1
},
{
  "Name": "Query Parquet solution data",
  "TaskType": "Post",
  "Files": [
    { "Path": "Team/User/**", "Version": null }
  ],
  "Arguments": "python3.12 query_imp_data.py",
  "ContinueOnError": true,
  "ExecutionOrder": 2
}
]

```

To confirm the active versions, run:

```

python3 --version
python3.12 --version

```

► Matching pip to the interpreter

Each interpreter has its own pip and its own site-packages. Using the wrong one is a common cause of a package being installed in the image but missing when the script imports it. Use the pip command that matches the interpreter running your script:

Command	Installs into
<code>pip</code> , <code>pip3</code> , <code>pip3.10</code>	Python 3.10
<code>pip3.12</code> , <code>python3.12 -m pip</code>	Python 3.12

Both interpreters come with `pip`, `wheel`, `setuptools`, and `duckdb` already available.

► How requirements.txt is handled

Only one `requirements.txt` file is supported per simulation. It is installed into **both** interpreters when the per-simulation image is built. The two installs are handled differently:

Interpreter	Install behavior	If the install fails
Python 3.10	Required	The container build fails and the simulation does not run.
Python 3.12	Best effort	A warning is logged in the build output and the simulation continues. Packages may be missing when a <code>python3.12</code> task imports them.

The Python 3.12 install is best-effort so that a package without Python 3.12 support cannot break simulations that only use Python 3.10.

i NOTE: A package that requires a newer Python version must include an environment marker, otherwise the required Python 3.10 install step will fail the container build. For example:

```
plexosdb>=0.1.0; python_version >= "3.11"
```

Without the marker, pip attempts to install the package under Python 3.10, cannot satisfy the requirement, and the build stops before the simulation starts.

► Confirming the version at run time

To confirm which interpreter a task used, print it from within the script. This is useful for diagnosing a package that imports locally but not in the cloud.

```
import platform
import sys

print(f"python_version: {platform.python_version()}")
print(f"executable: {sys.executable}")
```

Alternatively, add a task that logs both interpreters before anything else runs:

```
{
  "Name": "Log available Python versions",
  "TaskType": "Pre",
  "Arguments": "bash -c 'python3 --version; python3.12 --version'",
  "ContinueOnError": true,
  "ExecutionOrder": 0
}
```

NOTE: The `python3` command continues to resolve to Python 3.10, so existing task definitions do not need to change. Moving a script to `python3.12` is optional and can be done one task at a time after the script is validated on Python 3.12.

File System

The Linux container consists of two key directories:

1. **/simulation**
2. This serves as the root directory for study data, including XML files, time-series data, and other artifacts.
3. It is Read/Write for pre-simulation tasks but is Read-Only for post-simulation tasks to maintain data integrity.
4. **/output**
5. This is designated for user-generated data.
6. Any files remaining in this directory at the end of the simulation are automatically uploaded as part of the solution.

Environment Variables

The following environment variables set up system paths and values that assist tasks during execution:

Environment variable	Description
<code>tenant_id</code>	Stores your tenant ID.
<code>simulation_id</code>	Stores the current simulation ID.
<code>study_id</code>	Stores the current study ID.
<code>execution_id</code>	Stores the current execution ID.
<code>simulation_path</code>	Specifies the root path for PLEXOS XML and study files. It is read-only for post-simulation tasks.
<code>output_path</code>	Defines the working directory for scripts. Files remaining here upload automatically.
<code>cloud_cli_path</code>	Stores the full path to the PLEXOS Cloud CLI tool and Cloud Python SDK toolkit.
<code>auth_path</code>	Represents a file containing the raw user access token. Use it for automation workflows.
<code>duck_db_path</code>	Defines the default DuckDB project path for solution-data queries.
<code>directory_map_path</code>	Contains a JSON manifest for input and solution directories, including IDs.

Using standard formatting, common variables like **tenant_id**, **simulation_id**, **study_id**, and **execution_id** can also be used in command parameters. They are resolved at script runtime.

Examples

1. Logging Environment Variables

```
{
  "Name": "Printing variables to log",
  "TaskType": "Post",
  "Arguments": "echo Tenant: {tenant_id} Simulation: {simulation_id} Study:
{study_id} Execution: {execution_id}",
  "ContinueOnError": true,
  "ExecutionOrder": 1
}
```

2. Retrieving Environment Variables in Python

```
import os

# Get environment variables for simulation, output, and duckdb file paths.
simulation_path = os.environ.get("simulation_path", "/simulation")
output_path = os.environ.get("output_path", "/output")
```

Task Output

Any files remaining in the **/output** directory are automatically uploaded at the end of the simulation. These artifacts can be downloaded using the CLI as they will not appear on Cloud Web.

Examples

1. Listing Solution Files Using CLI \

```
plexos-cloud solution files list --solutionId b8737512-0375-439f-9260-
9db7f8d77ec0 --type TaskArtifacts
```

2. Downloading Files Using CLI \

```
pxc solution files download --solutionId b8737512-0375-439f-9260-
9db7f8d77ec0 --type TaskArtifacts -d c:\Path\ --overwrite
```


Executing Tasks on the Cloud

The CLI supports enqueueing simulations with tasks by providing paths to DataHub artifacts and passing arguments in the JSON body.

The **SimulationTasks** property in the JSON request defines tasks. Each task includes the following properties:

1. **Name:** A descriptive name for the task. This appears in logs and on the Cloud Web after the simulation.
2. **TaskType:** Pre or Post - Pre executes before the simulation engine, while Post executes after the engine and ETL processes.
3. **Files:** Defines the input files for the task.
4. **Path:**
5. **Explicit:** Full path to the required file.
6. **Wildcard:** A globbing pattern to download one or more files.
7. **Version:**
8. **Null:** Downloads the latest available version.
9. **Explicit:** Downloads a specific numeric version.
10. **SnapshotDate:** Downloads files matching the path as of a specified date.
11. **Arguments:** The command-line argument for executing the script. When using Python, python3 must precede the script name. Additional arguments are supported.
12. **ExecutionOrder:** The order in which tasks are executed. Users must specify an explicit and unique order to avoid ambiguity.

Examples

1. **Configuring Files and Running a Python Script **

```
{
  "Name": "DESCRIPTIVE NAME OF YOUR TASK",
  "TaskType": "Post",
  "Files": [
    {
      "Path": "Path/SubPath/My_Python_File.py",
      "Version": 1
    },
    {
      "Path": "Path/OtherPath/**",
      "SnapshotDate": "2025-01-01"
    },
    {
      "Path": "Path/SubPath/requirements.txt",
      "Version": null
    }
  ],
  "Arguments": "python3 My_Python_File.py arg1 arg2",
  "ContinueOnError": true,
  "ExecutionOrder": 0
}
```

2. Defining Multiple Simulation Tasks \

```

{
  "StudyId": "5ec77945-2b44-496d-8e0e-57fb68402a6a",
  "ChangeSetId": "5ec77945-2b44-496d-8e0e-57fb68402a6a",
  "Models": "YOUR MODEL NAME",
  "SimulationOptions": {
    "Locale": 1033,
    "SimulationTasks": [
      {
        "Name": "Writing membership data to disk for post simulation analysis",
        "TaskType": "Pre",
        "Files": [
          {
            "Path": "Team/User/query_write_memberships.py",
            "Version": 1
          },
          {
            "Path": "Team/User/**",
            "Version": null
          }
        ],
        "Arguments": "python3 query_write_memberships.py",
        "ContinueOnError": true,
        "ExecutionOrder": 1
      },
      {
        "Name": "Setting query context using CloudCLI",
        "TaskType": "Post",
        "Arguments": "plexos-cloud solution query configure-views --model-name 'YOUR MODEL NAME'",
        "ContinueOnError": true,
        "ExecutionOrder": 2
      },
      {
        "Name": "Post Simulation querying of parquet data",
        "TaskType": "Post",
        "Files": [
          {
            "Path": "Team/User/**",
            "Version": null
          }
        ],
        "Arguments": "python3 query_imp_data.py",
        "ContinueOnError": true,
        "ExecutionOrder": 3
      },
      {
        "Name": "Delete Post simulation script artifacts",

```

```
"TaskType": "Post",
"Arguments": "python3 clean_workflow_files.py",
"ContinueOnError": true,
"ExecutionOrder": 4
}
],
},
"SimulationData": {
  "Uri": "https://studies-api-
aquila.energyexemplar.com/1.0/downloads/studies/5ec77945-2b44-496d-
8e0e-57fb68402a6a/changesets/5ec77945-2b44-496d-8e0e-
57fb68402a6a/input-data",
  "Type": "ChangesetDatabase"
},
"SimulationEngine": {
  "EngineId": "52b42700-3d98-46ca-8ed6-a544d9129281",
  "Version": "10.000.003.20515"
},
"Source": "Automation",
"Priority": 1,
"RequestedCpuCores": 1,
"MinimumMemoryInGb": 4.0
}
```

3. Querying SQLite Data Using DuckDB in Python \

```

import os

import duckdb

def export_sqlite_query_to_csv() -> None:
    try:
        simulation_path = os.environ.get("simulation_path", "/simulation")
        output_path = os.environ.get("output_path", "/output")

        database_file_path = os.path.join(simulation_path, "reference.db")
        memberships_file_path = os.path.join(output_path,
        "memberships_data.csv")

        with duckdb.connect(database_file_path) as con:
            con.execute("INSTALL sqlite;")
            con.execute("LOAD sqlite;")

            print(f"Configuring DuckDB to use: {database_file_path}")
            con.execute(f"ATTACH '{database_file_path}' (TYPE SQLITE);")
            con.execute("USE reference;")

            query_text = """
            SELECT
                cl1.Name AS parent_class,
                cl2.Name AS child_class,
                col.Name AS collection,
                obj1.Name AS parent_object,
                obj2.Name AS child_object,
                " AS subcollection_name
            FROM t_membership mem
            INNER JOIN t_object obj1 ON obj1.object_id = mem.parent_object_id
            INNER JOIN t_object obj2 ON obj2.object_id = mem.child_object_id
            INNER JOIN t_collection col ON col.collection_id = mem.collection_id
            INNER JOIN t_class cl1 ON cl1.class_id = mem.parent_class_id
            INNER JOIN t_class cl2 ON cl2.class_id = mem.child_class_id
            """

            con.execute(
                f"COPY ({query_text}) TO '{memberships_file_path}' WITH (HEADER,
DELIMITER ');"
            )
            print(f"Membership information written to {memberships_file_path}")
    except Exception as e:
        print("Exporting SQLite data to CSV failed due to an exception:")
        print(e)

```

```
if __name__ == "__main__":  
    export_sqlite_query_to_csv()
```

Using Energy Exemplar Cloud CLI or Cloud Python SDK

The latest Cloud CLI is available by default for all tasks. The CLI binary is configured in **/usr/local/bin**, allowing users to run commands without specifying a full path. If the **Cloud Python SDK** is used, no additional setup is required.

Authentication

The user credentials that enqueue a simulation is automatically used for all CLI commands. Authentication against the current environment is already handled.

Example

1. Listing Recent Simulations Using CLI

```
{  
  "Name": "List last 10 simulations",  
  "TaskType": "Post",  
  "Arguments": "plexos-cloud simulation list --top 10 --orderBy LastUpdatedAt --  
descending",  
  "ContinueOnError": true,  
  "ExecutionOrder": 0  
}
```

Accessing or Querying Model Data

DuckDB enables users to generate views, making it easier to query Parquet files on disk. To simplify this process, a new CLI command extends this functionality, reducing the need for users to locate files on the disk manually.

Directory Mapping File

A special directory mapping file stores paths to commonly used files, eliminating the need for manual directory inspection. This JSON-based file contains model names and well-defined paths to the Simulation Root (solution directory) and Output paths. The file location persists in the **directory_map_path** variable.

Examples

1. Directory Mapping JSON File \

```
[
  {
    "Name": "Simulation",
    "Path": "/simulation",
    "Id": "a3457ef3-4b7a-4312-81a5-8cae550b9de2"
  },
  {
    "Name": "Output",
    "Path": "/output"
  },
  {
    "Name": "MODEL NAME",
    "Path": "/simulation/Model MODEL NAME Solution",
    "ParquetPath": "/simulation/Model MODEL NAME
Solution/version2/ParquetUploads",
    "Id": "8d28ee05-8618-489b-b38a-52ed00dba4ab"
  }
]
```

2. Configuring SQL Views Using CLI \

```
# Use CLI to create or update SQL views that look at solution data on disk.  
plexos-cloud solution query configure-views --name "YOUR MODEL NAME"
```

3. JSON Task Definition for Configuring Views \

```
{  
  "Name": "Configure query views for YOUR MODEL NAME",  
  "TaskType": "Post",  
  "Arguments": "python3 configure_duck.py 'YOUR MODEL NAME' False",  
  "ContinueOnError": false,  
  "ExecutionOrder": 3  
}
```

4. Script to parse the `directorymapping.json` file produced on every run


```
import json
import os
import sys

import duckdb

def read_path_from_mapping_file(mapping_file_path: str, model_name: str) -> str | None:
    """Read the ParquetPath for the given model name."""
    with open(mapping_file_path, "r") as file:
        data = json.load(file)

    for item in data:
        if item.get("Name") == model_name:
            return item.get("ParquetPath")

    return None

def find_subdirectories(root_dir: str) -> list[str]:
    """Find all subdirectories for the given root directory."""
    subdirectories: list[str] = []

    for dirpath, dirnames, _filenames in os.walk(root_dir):
        for dirname in dirnames:
            subdirectories.append(os.path.join(dirpath, dirname))

    return subdirectories

def configure_duck_views(model_name: str, verbose_log: bool = False) -> None:
    """Configure DuckDB views for the given model name."""
    try:
        duck_file_path = os.environ.get("duck_db_path",
        "/output/solution_views.ddb")
        print(f"Setting up DuckDB views using {duck_file_path}")

        mapping_file_path = os.environ.get(
            "directory_map_path",
            "/simulation/directorymapping.json",
        )
        model_directory = read_path_from_mapping_file(mapping_file_path,
        model_name)

        if model_directory is None:
            raise Exception(f"Unable to find output for model name {model_name}")
```

```

print(f"Solution data found for {model_name}: {model_directory}")
directories = find_subdirectories(model_directory)

with duckdb.connect(duck_file_path) as con:
    for item in directories:
        view_name = item.replace(model_directory, "").replace("/",
        "").replace("\\", "")
        path = os.path.join(item, "**", "*.parquet")

        if "datadataFileId=" in view_name:
            break

        view_command = (
            f"CREATE OR REPLACE VIEW {view_name} AS SELECT * FROM
            '{path}';"
        )

        if verbose_log:
            print(view_command)

        con.execute(view_command)

        if verbose_log:
            con.sql(f"SELECT * FROM {view_name} LIMIT 2;").show()
except Exception as e:
    print("Configuring DuckDB views failed due to an exception:")
    print(e)

if __name__ == "__main__":
    if len(sys.argv) > 1:
        model_name = sys.argv[1]
        verbose_log = len(sys.argv) > 2 and sys.argv[2].lower() in ("1", "true")
        configure_duck_views(model_name, verbose_log)
    else:
        print("Model Name parameter is required.")

```

5. File Paths must be specified in the required directorymapping.json file

```

{
  "Name": "Your Model Name",
  "Path": "C:\\Sim\\Model YOUR MODEL NAME Solution",
  "ParquetPath": "C:\\Sim\\Model YOUR MODEL NAME
  Solution\\version2\\ParquetUploads"
}

```

6. The following views are created or replaced. If multiple models are executed, you must run the script above with the model name parameter, configuring the paths to the model solution data again.

```
CREATE OR REPLACE VIEW aggregatedseries AS
SELECT * FROM 'C:\Sim\Model YOUR MODEL NAME
Solution\version2\ParquetUploads\Aggregatedseries\*_*.parquet';
```

```
CREATE OR REPLACE VIEW attribute AS
SELECT * FROM 'C:\Sim\Model YOUR MODEL NAME
Solution\version2\ParquetUploads\Attribute\*_*.parquet';
```

```
CREATE OR REPLACE VIEW attributedata AS
SELECT * FROM 'C:\Sim\Model YOUR MODEL NAME
Solution\version2\ParquetUploads\Attributedata\*_*.parquet';
```

```
CREATE OR REPLACE VIEW category AS
SELECT * FROM 'C:\Sim\Model YOUR MODEL NAME
Solution\version2\ParquetUploads\Category\*_*.parquet';
```

```
CREATE OR REPLACE VIEW class AS
SELECT * FROM 'C:\Sim\Model YOUR MODEL NAME
Solution\version2\ParquetUploads\Class\*_*.parquet';
```

```
CREATE OR REPLACE VIEW classgroup AS
SELECT * FROM 'C:\Sim\Model YOUR MODEL NAME
Solution\version2\ParquetUploads\Classgroup\*_*.parquet';
```

```
CREATE OR REPLACE VIEW collection AS
SELECT * FROM 'C:\Sim\Model YOUR MODEL NAME
Solution\version2\ParquetUploads\Collection\*_*.parquet';
```

```
CREATE OR REPLACE VIEW config AS
SELECT * FROM 'C:\Sim\Model YOUR MODEL NAME
Solution\version2\ParquetUploads\Config\*_*.parquet';
```

```
CREATE OR REPLACE VIEW customcolumn AS
SELECT * FROM 'C:\Sim\Model YOUR MODEL NAME
Solution\version2\ParquetUploads\Customcolumn\*_*.parquet';
```

```
CREATE OR REPLACE VIEW data AS
SELECT * FROM 'C:\Sim\Model YOUR MODEL NAME
Solution\version2\ParquetUploads\Data\*_*.parquet';
```

```
CREATE OR REPLACE VIEW fullkeyinfo AS
SELECT * FROM 'C:\Sim\Model YOUR MODEL NAME
Solution\version2\ParquetUploads\Fullkeyinfo\*_*.parquet';
```

```
CREATE OR REPLACE VIEW membershipinfo AS
SELECT * FROM 'C:\Sim\Model YOUR MODEL NAME
Solution\version2\ParquetUploads\Membershipinfo\*_*.parquet';
```

```
CREATE OR REPLACE VIEW memoobject AS
SELECT * FROM 'C:\Sim\Model YOUR MODEL NAME
Solution\version2\ParquetUploads\Memoobject\*_*.parquet';
```

```
CREATE OR REPLACE VIEW object AS
SELECT * FROM 'C:\Sim\Model YOUR MODEL NAME
Solution\version2\ParquetUploads\Object\*_*.parquet';
```

```
CREATE OR REPLACE VIEW objectmeta AS
SELECT * FROM 'C:\Sim\Model YOUR MODEL NAME
Solution\version2\ParquetUploads\Objectmeta\*_*.parquet';
```

```
CREATE OR REPLACE VIEW period AS
SELECT * FROM 'C:\Sim\Model YOUR MODEL NAME
Solution\version2\ParquetUploads\Period\*_*.parquet';
```

Task Examples

1. Uploading Files to DataHub Using CLI \

```
{
  "Name": "Uploading report files via CLI",
  "TaskType": "Post",
  "Arguments": "plexos-cloud DataHub upload --local-folder /output --remote-
folder Team/User/Reports --pattern Report_*.csv",
  "ContinueOnError": true,
  "ExecutionOrder": 5
}
```

2. Enqueuing a New Simulation Using CLI \

```
{
  "Name": "Enqueue new simulation",
  "TaskType": "Post",
  "Arguments": "plexos-cloud simulation enqueue --file /simulation/newsim.json -f
table",
  "ContinueOnError": true,
  "ExecutionOrder": 6
}
```

Script Examples

1. Querying Parquet Data Using DuckDB \

```
import os

import duckdb

def export_parquet_to_csv(output_csv: str) -> None:
    """Query Parquet views and write the result to a CSV file."""
    try:
        output_path = os.environ.get("output_path", "/output")
        duck_file_path = os.environ.get(
            "duck_db_path",
            os.path.join(output_path, "solution_views.ddb"),
        )
        print("Starting DuckDB")

        with duckdb.connect(duck_file_path) as con:
            membership_info_data = "SELECT * FROM membershipinfo"
            con.execute(
                f"COPY ({membership_info_data}) TO '{output_csv}' WITH (HEADER,
DELIMITER '');"
            )
            con.sql(membership_info_data).show()
    except Exception as e:
        print("Exporting Parquet data to CSV failed due to an exception:")
        print(e)

if __name__ == "__main__":
    output_path = os.environ.get("output_path", "/output")
    output_csv = os.path.join(output_path, "my_data.csv")
    export_parquet_to_csv(output_csv)
```

2. Uploading Solution Data to DataHub Manually \

```
python3 upload_to_DataHub.py "No Carbon Price or Flow Constraint"
```

```
import json
import os
import sys

from eecloud.cloudsdk import CloudSDK
from eecloud.models import *

class ModelData:
    def __init__(self, id: str, parquet_path: str):
        self.id = id
        self.parquet_path = parquet_path

def read_path_from_mapping_file(mapping_file_path: str, model_name: str) ->
ModelData | None:
    with open(mapping_file_path, "r") as file:
        data = json.load(file)

    for item in data:
        if item.get("Name") == model_name:
            return ModelData(item.get("Id"), item.get("ParquetPath"))

    return None

def main() -> None:
    try:
        cli_path = os.environ.get("cloud_cli_path")
        pxc = CloudSDK(cli_path=cli_path)

        model_name = ""
        if len(sys.argv) > 1:
            model_name = sys.argv[1]

        if not model_name:
            raise Exception("Model Name parameter is required.")

        mapping_file_path = os.environ.get(
            "directory_map_path",
            "/simulation/directorymapping.json",
        )
        map_data = read_path_from_mapping_file(
            mapping_file_path=mapping_file_path,
            model_name=model_name,
        )
```

```

if map_data is None:
    raise Exception(f"Unable to find mapping data for model {model_name}")

simulation_id = os.environ.get("simulation_id")
execution_id = os.environ.get("execution_id")
remote_path = f"Eagles/Solutions/{execution_id}/{map_data.id}"

print(f"Uploading solution {map_data.id} for simulation {simulation_id}
using SDK")
pxc.DataHub.upload(
    local_folder=map_data.parquet_path,
    remote_folder=remote_path,
    glob_patterns="**/*.parquet",
    is_versioned=False,
    print_message=True,
)
except Exception as e:
    print("Uploading solution data using CLI failed:")
    print(e)
finally:
    print("done")

if __name__ == "__main__":
    main()

```

NOTE: The examples and commands provided in this document are subject to change. Please refer to the latest Energy Exemplar documentation or CLI updates for any modifications or enhancements**. **

Previous
 < Simulation Monitoring
 Dashboard

Next
 Manage Study Level >
 Simulation Preferences

Last updated 9 days ago

Was this helpful?



Additional Resources

[Service Level Agreement \(SLA\)](#)

[PLEXOS Cloud Security and Compliance](#)

© 2026 Energy Exemplar. All rights reserved.